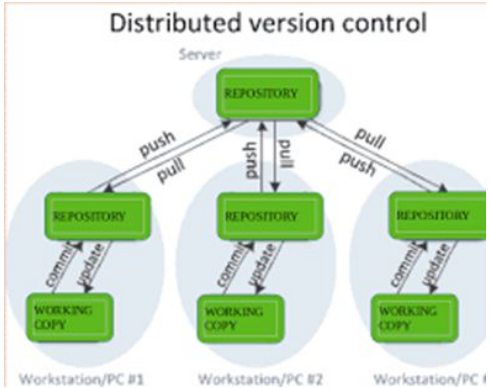
USO DE GIT

Control de Versiones y trabajo colaborativo

Introducción a GIT

En esta presentación, veremos el sistema de control de versiones distribuido que permite realizar un seguimiento de los cambios en el código y colaborar con otros desarrolladores



Que es GIT



- Un sistema de control de versiones.
- Permite realizar un seguimiento de los cambios en el código
- Ayuda a colaborar con otros desarrolladores
- Es gratuito y de código abierto.

Existen Varias en el mercado todas on-line y con diferentes modos de contratación.

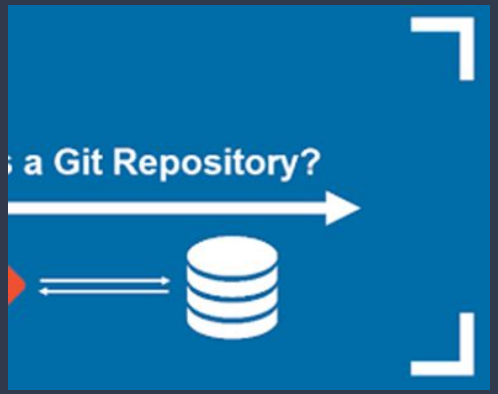
GITHub
BitBuket
AzureRepos
GITLab
GITKraken

¿Por qué usar GIT?



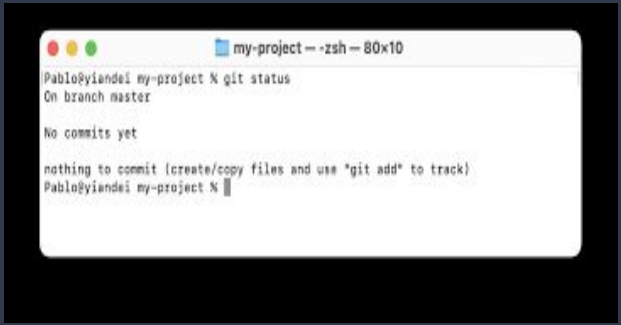
- Realiza un seguimiento de los cambios en tu código
- Colabora con otros desarrolladores
- Evita la pérdida de datos
- Vuelve a versiones anteriores de tu código
- Escala para proyectos grandes

Cómo Funciona GIT



- Repositorio local: copia local de tu código
- Repositorio remoto: copia central de tu código
- Commits: instantáneas de tu código en un punto del tiempo
- Rama: línea de desarrollo independiente
- Push: envía cambios locales al repositorio remoto
- Pull: descarga cambios del repositorio remoto a tu repositorio local
- Merge: junta ramas en tu repositorio

Comando Básicos de GIT



git init: inicializa un nuevo repositorio

git add: agrega archivos al área de preparación

git commit: crea un nuevo commit

git status: muestra el estado de tu repositorio

git branch: muestra las ramas de tu repositorio

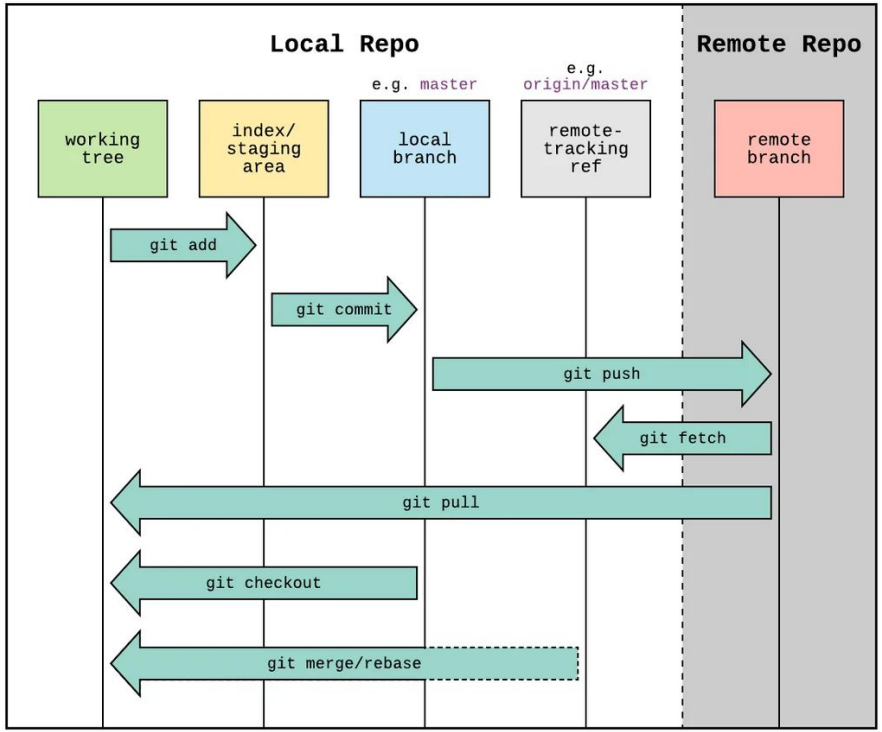
git checkout: cambia a una rama diferente

git merge: fusionar una rama en tu rama principal

git push: envía cambios al repositorio remoto

git pull: descarga cambios del repositorio remoto

Flujo de Trabajo



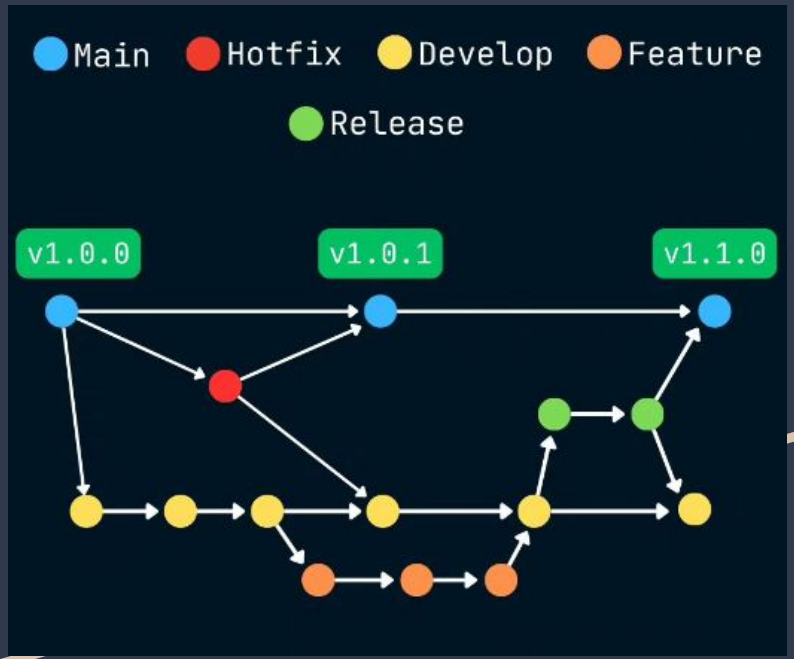
- Crea una rama por cada desarrollador como mínimo.
- Trabaja en tu función en la rama
- Haz commits con frecuencia para realizar un seguimiento de tu progreso
- Cuando tu función esté lista, fusiona la rama en tu rama principal
- Empuja los cambios al repositorio remoto

Beneficios



- **Organiza tu código:** Git te ayuda a organizar tu código y realizar un seguimiento de los cambios.
- **Colabora con otros:** Git te permite colaborar con otros desarrolladores en proyectos de código.
- **Evita la pérdida de datos:** Git te ayuda a evitar la pérdida de datos al realizar un seguimiento de las revisiones de tu código.
- **Vuelve a versiones anteriores:** Git te permite volver a versiones anteriores de tu código si es necesario.
- **Escala para proyectos grandes:** Git es escalable para proyectos grandes y complejos.

Patrones de uso



Cuando versionamos el código con Git, deberíamos usar algún "patrón" para poder ser ordenado y estructurado.

De esta manera evitamos subir código roto a producción y generar errores en nuestra aplicación/página web

Esto patrones se denominan GITFLOW o flujo de trabajo basado en git.

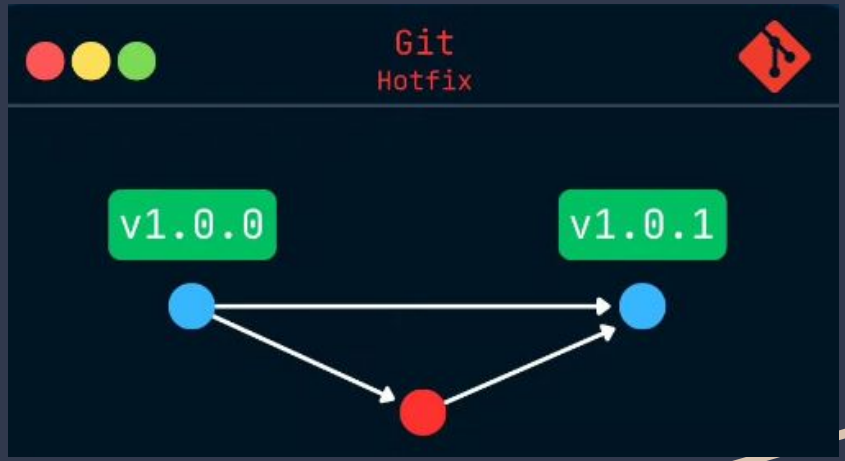
GITFlow - main



La rama **main** solo tiene codigo que esta en producción .

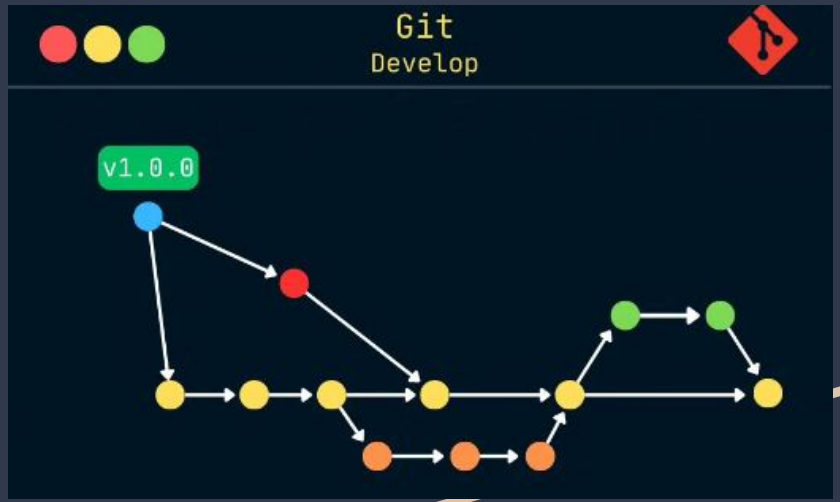
Los únicos commits a main son merges de ramas **release** y hotfix

GITFlow - hotfix



Las ramas **hotfix** se usan para crear arreglos de emergencias para problemas de **producción**. Se “Saca” una rama desde **main** y luego se mergea en esta última

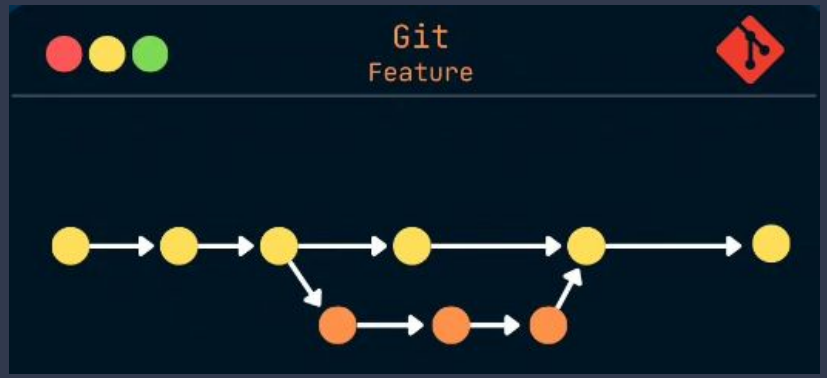
Develop



Develop es una de las ramas principales en el flujo de trabajo.

Las ramas **feature** y **release** se sacan desde la rama **develop** y cuando se terminan se mergean nuevamente

Feature



Las ramas **feature** “se sacan” desde **develop**.

En ellas se van a desarrollar las nuevas características del proyecto, una vez finalizadas se mergean en **develop**

Release



Cuando es hora de subir una nueva versión, “sacamos” una rama **release** desde **develop**.

si esa rama no tiene errores será la que se mergee en **main**

Práctica

- Crear repositorio en GitHub
- Descargar Git Bash
- Realizar el TP 1 de campus



Muchas Gracias